

Aave Labs - Stable Vaults - Extension 3

Final report

Prepared by: Josselin Feist (josselin@seceureka.com)

July 6th, 2026

Prepared For: Aave Labs

Executive Summary	3
Overview	3
Security findings	4
Lack of fee bounds on bridge operations	5
Summary	5
Detailed description	5
Recommendations	5
Status	5
Out-of-order event risk in bridgeTokens	6
Summary	6
Detailed description	6
Recommendations	7
Status	7
Appendix A: May 18th review result	8
Mitigation tradeoff strategy	8
Appendix B: Dual accounting risks	10
Recommendations	11

Executive Summary

Overview

During the week of May 18, we performed a one-week review of the Stable Vault contracts, covering the changes made to the contracts since the last review. We focused on the changes made to the src directory, working from commit c179e978c9deb5517686d35915ef54c545745aa.

The results of our review are presented in [Appendix A](#).

Moreover, additional ad hoc work on the dual accounting risk was conducted during the week of June 8 and is described in [Appendix B](#).

This report is part of a multiple stages review of the Stable Vault (4/4).

Security findings

ID	Title	Severity	Impact	Likelihood	Status
1	Lack of fee bounds on bridge operations	<i>Low</i>	<i>Low</i>	<i>Low</i>	<i>Acknowledged</i>
2	Out-of-order event risk in bridgeTokens	<i>Informational</i>	<i>Informational</i>	<i>Informational</i>	<i>Fixed</i>

Lack of fee bounds on bridge operations		
<i>Impact: Low</i>	<i>Likelihood: Low</i>	<i>001</i>

Summary

The lack of fee bounds on the bridge's operations can cause the users to pay more fees than expected.

Detailed description

transferFrom is used to pay the fee in CcipAdapter and AdiAdapter:

```

        if (estimatedFeeAmount > 0) {
            IERC20(ccipFeeParams.feeToken).safeTransferFrom(feePayer,
address(this), estimatedFeeAmount);
        }

```

Figure 1: CcipAdapter.sol#L214-L216

There is no per-transaction bound on the maximum fees. As a result the user might end up paying up to their approved token amount.

The amount of fees to be paid is determined by the fee quoter. This component does not usually provide the same guarantees as the underlying bridge. For example, the security of the fee quoter on CCIP relies on a mix of configuration and price feeds, making it less secure than the DON architecture.

As users tend to pre-approve unlimited tokens, this could lead to a significant impact on users if the fee quoter(s) were to be compromised.

Recommendations

Add a maximum fee amount per transaction on the bridge adapters.

Status

The Aave Labs acknowledged and decided to document it as an accepted risk.

In addition, Aave Labs mentioned that CCIP would be used for funds bridging, while ADI would be used for data-only messages.

Out-of-order event risk in bridgeTokens

Impact: Informational

Likelihood: Informational

002

Summary

Reentrancy during the fee refund can allow an attacker to emit the event in bridgeTokens in an incorrect order.

Detailed description

When paying fees with the native token, a refund of the excess native asset sent might occur:

```
function _triggerNativeFeeRefund(address feePayer, uint256 excessFee)
internal {
    (bool callSucceeded,) = payable(feePayer).call{value:
excessFee}("");
    require(callSucceeded, Errors.NativeTransferFailed());
    emit FeeRefunded(feePayer, Constants.NATIVE_CURRENCY, excessFee);
}
```

Figure 1: CcipAdapter.sol#L333-L337

This call allows for an arbitrary callback to the fee payer, allowing it to re-enter into the contract.

This operation happens before the TokensBridged event is emitted:

```
IChainGateway(CHAIN_GATEWAY).sendBridgeIouTokenMessageWithFeePayer{value
: msg.value}(
    destinationChainId,
    iouTokenRecipient,
    iouTokenAmountRay,
    bridgeAdapter,
    msg.sender,
    payloadExecutionGasLimit,
    bridgeAdapterData
);

emit TokensBridged(destinationChainId, iouTokenRecipient,
```

```
iouTokenAmountRay) ;
```

Figure 2: `IouTokenManager.sol`#L118-L128

An attacker using the reentrancy could therefore make the `TokensBridged` events be emitted in an incorrect order.

Note: the same can happen on `pushFundsToChain` and `pushFundsToAccountingChain`, though these functions are protected by the `restricted` modifier.

Recommendations

Consider adding the `nonReentrant` modifier in `bridgeTokens`, `pushFundsToChain` and `pushFundsToAccountingChain`. Ensure that the off-chain monitoring does not rely on the event order.

Status

Fixed with [PR328](#).

Appendix A: May 18th review result

The review identifies two new issues ([issue 1](#) and [2](#)), and highlighted the following potential improvements:

- **Consider if a “per asset” rate limit should be added on the withdrawal operation.** This would provide more granularity on the outflow of tokens, and allow stricter outflow rules to be created for tokens that are considered more robust.
- **Document that the swapper can bypass the transfer of leftover fees from SlippageCoverageVault.** SlippageCoverageVault exists only as an operational helper, and its underlying protection should not be considered as strictly imposed.
- **Consider checking the return value of EnumerableSet.add/remove (ex: in Allocator and AssetRegistry).** These functions return a boolean value indicating whether the operation was a success or not.

Mitigation tradeoff strategy

We noted that some of the mitigations create tradeoffs between additional safeguards and complexity. For example, the try/catch and EIP-150 handling made on the price Oracle:

```
// Wrap the adapter call in a try-catch so a single adapter
revert (paused feed, sequencer downtime, etc.)
// does not block downstream consumers – the catch returns zero,
mirroring the stale-response branch.
try IPriceOracleAdapter(oracleAdapter).getPrice(asset) returns (
    IPriceOracleAdapter.OracleResponse memory response
) {
    return response.isStale ? 0 :
_capToMaxPrice(response.priceRay);
} catch {
    // The "all but one 64th" gas check re-reverts when the catch
fired from out-of-gas, so a healthy feed
    // cannot be silently induced to return zero. See EIP-150 for
more details.
    if (gasleft() <= gasBefore / 64) {
        revert InsufficientGasForExternalCall();
    }
    return 0;
}
```

Figure 1: PriceOracle.sol#L114-L127

This mitigation adds low-level logic and a maintenance burden, while offering limited protection. Some of the listed risks cannot occur. Moreover if `latestRoundData` starts reverting, `distrustAsset / setOracleAdapterForAsset` will need to be called, making the benefit of `try/catch` only relevant for the time period between the reverting and the configuration update.

We recommend to ask these questions when adding further mitigations:

- **Does the mitigation prevent a security risk or failure mode?**
- **Does the mitigation involve complex code?**
- **Is there an existing unlocking path?**

For this example, the protection prevents a low likelihood failure mode, requires low-level logic, and already has an existing unlocking path. In that context, the cost of the mitigation might outweigh the benefit.

Note that a similar try/catch strategy exists on the balance Oracle, however due to the non-standard operation performed for the balance decoding, the try/catch is justified.

For the long term, consider maintaining a mitigation decision matrix for safeguards added to the system. For each mitigation, the matrix should track the failure mode it protects against, the assumptions it relies on, the trusted actors involved, the expected behavior when it triggers, any bypass or recovery path, and the complexity or operational cost it introduces.

This matrix should be kept aligned with the Primer document, so that protocol invariants and assumptions remain up to date as safeguards accumulate. Over time, some failure modes become harder to reason about because the relevant behavior is no longer only in the direct code, but also in the surrounding policy, oracle, rate-limit, bridge, and pause logic. For example, evaluating what can block withdrawals requires understanding each mitigation that may affect the withdrawal path.

Keeping this matrix up to date would make security tradeoffs easier to review, clarify which mitigations are security-critical versus operational defense-in-depth, and reduce the risk that future changes accidentally weaken an existing safeguard.

Appendix B: Dual accounting risks

It was observed that several informational issues stemmed from the dual accounting nature of the stable vault position, where the total asset amount is tracked as shares, while the guaranteed original deposit amount is tracked as a ray amount (`originalDepositRay`).

```
struct UserPosition {  
    uint256 originalDepositRay;  
    uint256 subVaultId;  
    uint256 shares;  
}
```

Figure 1: UserPosition structure

When a position has $\text{shares} * \text{rate} < \text{originalDepositRay}$, the following can occur:

- Calling `deposit`, and `transfer(..., balanceOf(user))` in the same transaction can revert
 - As the amount of shares required to transfer `balanceOf(user)` can be higher than the position's shares
- Calling `deposit`, and `requestWithdrawal(..., balanceOf(user), ...)` in the same transaction can revert
 - As the amount of shares required to withdraw `balanceOf(user)` can be higher than the position's shares
- `transferAll` can transfer fewer shares than anticipated.
 - As the source original deposit is fully transferred, fewer shares are issued to the destination. This leads the destination accounting to receive fewer shares than implied by the transferred guarantee
- `_getActiveSubVaultsObligations` can understate the true obligation
 - As the strict per-user obligation is $\text{sum}(\max(\text{userShares} * \text{userRate}, \text{userOriginalDepositRay}))$, while the current aggregate floor is $\max(\text{sum}(\text{subVaultTotalShares} * \text{subVaultRate}), \text{globalOriginalDepositsRay})$.
 - The strict per-user value can be higher because interest from positions where $\text{userShares} * \text{userRate} > \text{userOriginalDepositRay}$ can offset deficits from positions where $\text{userShares} * \text{userRate} < \text{userOriginalDepositRay}$ in the aggregate formula.
 - The gap is bounded by $\text{sum}(\text{userOriginalDepositRay} - \text{userShares} * \text{userRate})$ over positions where $\text{userShares} * \text{userRate} < \text{userOriginalDepositRay}$.
 - This can make `claimSurplusInterest` overstate safely claimable surplus, allowing the treasury to claim more value than expected

```

function _getActiveSubVaultsObligations() internal view returns
(uint256) {
    uint256 activeSubVaultsObligations;
    uint256 activeSubVaultsCount =
$storage().activeSubVaultsIds.length;
    for (uint256 i = 0; i < activeSubVaultsCount; i++) {
        uint256 subVaultId = $storage().activeSubVaultsIds[i];
        // Round up the obligations to avoid understating
liabilities.
        activeSubVaultsObligations +=
$storage().subVaultById[subVaultId].totalShares
            .rayMulUp(_previewSubVaultConversionRate(subVaultId));
    }
    return Math.max(activeSubVaultsObligations,
$storage().globalOriginalDepositsRay);
}

```

Figure 2: _getActiveSubVaultsObligations

The divergence between $\text{shares} * \text{rate}$ and $\text{originalDepositRay}$ can be increased by

- deposit
- transfer / transferAll
- setUserRate

The increase is additive and scales roughly linearly with the share/rate or conversion/rate ratio, but not exponentially. In the tested scenario (20 years, min rate and 20% max rate), one round trip loses about 38-39 low-rate share wei. The increase is therefore small per operation.

As a result, while multiple issues exist, all of them have a minor impact.

Recommendations

Each individual issue can be fixed separately. However this would increase code complexity and not fix the root cause.

An alternative approach would be to change the dual accounting strategy.

One solution would be to adjust the $\text{originalDepositRay}$ to $\text{shares} * \text{rate}$ when $\text{originalDepositRay}$ exceeds $\text{shares} * \text{rate}$.

Another solution could be to track the original deposit not as a RAY amount, but as an index, representing the index for which the shares match the RAY amount. A position would therefore be represented as

- `originalDepositIndex`
- `subVaultId`
- `shares`

The original deposit in RAY would therefore be `originalDepositIndex * shares`. `originalDepositIndex` would be updated accordingly on every operation that updates the underlying guaranteed deposit. The invariant `originalDepositIndex <= currentConversionRate` would need to be preserved so the inferred principal does not exceed the share-backed value.

Such design would keep the guaranteed deposit attached to the shares, so transfers, withdrawals, and migrations move a consistent share/cost-basis pair.

One downside is that `originalDepositIndex` would be subject to precision loss, potentially reducing the guaranteed amount. However such precision loss might be acceptable.

We have not fully explored these designs, and they require additional consideration.